



# 20

---

## PHYSICAL DATABASE DESIGN AND TUNING

- ☛ What is physical database design?
- ☛ What is a query workload?
- ☛ How do we choose indexes? What tools are available?
- ☛ What is co-clustering and how is it used?
- ☛ What are the choices in tuning a database?
- ☛ How do we tune queries and view?
- ☛ What is the impact of concurrency on performance?
- ☛ How can we reduce lock contention and hotspots?
- ☛ What are popular database benchmarks and how are they used?
- ➡ **Key concepts:** Physical database design, database tuning, workload, co-clustering, index tuning, tuning wizard, index configuration, hot spot, lock contention, database benchmark, transactions per second

Advice to a client who complained about rain leaking through the roof onto the dining table: "Move the table."

.....Architect Frank Lloyd Wright

The performance of a DBMS on commonly asked queries and typical update operations is the ultimate measure of a database design. A DBA can improve performance by identifying performance bottlenecks and adjusting some DBMS parameters (e.g., the size of the buffer pool or the frequency of checkpointing) or adding hardware to eliminate such bottlenecks. The first step in achieving

good performance, however, is to make good database design choices, which is the focus of this chapter.

After we design the *conceptual* and *external* schemas, that is, create a collection of relations and views along with a set of integrity constraints, we must address performance goals through physical database design, in which we design the *physical* schema. As user requirements evolve, it is usually necessary to tune, or adjust, all aspects of a database design for good performance.

This chapter is organized as follows. We give an overview of physical database design and tuning in Section 20.1. The most important physical design decisions concern the choice of indexes. We present guidelines for deciding which indexes to create in Section 20.2. These guidelines are illustrated through several examples and developed further in Sections 20.3. In Section 20.4, we look closely at the important issue of clustering; we discuss how to choose clustered indexes and whether to store tuples from different relations near each other (an option supported by some DBMSs). In Section 20.5, we emphasize how well-chosen indexes can enable some queries to be answered without ever looking at the actual data records. Section 20.6 discusses tools that can help the DBA to automatically select indexes.

In Section 20.7, we survey the main issues of database tuning. In addition to tuning indexes, we may have to tune the conceptual schema as well as frequently used query and view definitions. We discuss how to refine the conceptual schema in Section 20.8 and how to refine queries and view definitions in Section 20.9. We briefly discuss the performance impact of concurrent access in Section 20.10. We illustrate tuning on our Internet shop example in Section 20.11. We conclude the chapter with a short discussion of DBMS benchmarks in Section 20.12; benchmarks help evaluate the performance of alternative DBMS products.

## 20.1 INTRODUCTION TO PHYSICAL DATABASE DESIGN

Like all other aspects of database design, physical design must be guided by the nature of the data and its intended use. In particular, it is important to understand the typical workload that the database must support; the workload consists of a mix of queries and updates. Users also have certain requirements about how fast certain queries or updates must run or how many transactions must be processed per second. The workload description and users' performance requirements are the basis on which a number of decisions have to be made during physical database design.

**Identifying Performance Bottlenecks:** All commercial systems provide a suite of tools for monitoring a wide range of system parameters. These tools, used properly, can help identify performance bottlenecks and suggest aspects of the database design and application code that need to be tuned for performance. For example, we can ask the DBMS to monitor the execution of the database for a certain period of time and report on the number of clustered scans, open cursors, lock requests, checkpoints, buffer scans, average wait time for locks, and many such statistics that give detailed insight into a *snapshot* of the live system. In Oracle, a report containing this information can be generated by running a script called UTLBSTAT.SQL to initiate monitoring and a script UTLBSTAT.SQL to terminate monitoring. The system catalog contains details about the sizes of tables, the distribution of values in index keys, and the like. The plan generated by the DBMS for a given query can be viewed in a graphical display that shows the estimated cost for each plan operator. While the details are specific to each vendor, all major DBMS products on the market today provide a suite of such tools.

To create a good physical database design and tune the system for performance in response to evolving user requirements, a designer must understand the workings of a DBMS, especially the indexing and query processing techniques supported by the DBMS. If the database is expected to be accessed concurrently by many users, or is a *distributed database*, the task becomes more complicated and other features of a DBMS come into play. We discuss the impact of concurrency on database design in Section 20.10 and distributed databases in Chapter 22.

### 20.1.1 Database Workloads

The key to good physical design is arriving at an accurate description of the expected workload. A workload description includes the following:

1. A list of queries (with their frequency, as a ratio of all queries / updates).
2. A list of updates and their frequencies.
3. Performance goals for each type of query and update.

For each query in the workload, we must identify

- Which relations are accessed.
- Which attributes are retained (in the SELECT clause).

- Which attributes have selection or join conditions expressed on them (in the WHERE clause) and how selective these conditions are likely to be.

Similarly, for each update in the workload, we must identify

- Which attributes have selection or join conditions expressed on them (in the WHERE clause) and how selective these conditions are likely to be.
- The type of update (INSERT, DELETE, or UPDATE) and the updated relation.
- For UPDATE commands, the fields that are modified by the update.

Remember that queries and updates typically have parameters, for example, a debit or credit operation involves a particular account number. The values of these parameters determine selectivity of selection and join conditions.

Updates have a query component that is used to find the target tuples. This component can benefit from a good physical design and the presence of indexes. On the other hand, updates typically require additional work to maintain indexes on the attributes that they modify. Thus, while queries can only benefit from the presence of an index, an index may either speed up or slow down a given update. Designers should keep this trade-off in mind when creating indexes.

## 20.1.2 Physical Design and Tuning Decisions

Important decisions made during physical database design and database tuning include the following:

### 1. Choice of indexes to create:

- Which relations to index and which field or combination of fields to choose as index search keys.
- For each index, should it be clustered or unclustered?

### 2. *Tuning the conceptual schema:*

- *Alternative normalized schemas:* We usually have more than one way to decompose a schema into a desired normal form (BCNF or 3NF). A choice can be made on the basis of performance criteria.
- *Denormalization:* We might want to reconsider schema decompositions carried out for normalization, during the conceptual schema design process to improve the performance of queries that involve attributes from several previously decomposed relations.

- *Vertical partitioning:* Under certain circumstances we might want to further decompose relations to improve the performance of queries that involve only a few attributes.
  - *Views:* We might want to add some views to mask the changes in the conceptual schema from users.
3. *Query and transaction tuning:* Frequently executed queries and transactions might be rewritten to run faster.

In parallel or distributed databases, which we discuss in Chapter 22, there are additional choices to consider, such as whether to partition a relation across different sites or whether to store copies of a relation at multiple sites.

### **20.1.3 Need for Database Tuning**

Accurate, detailed workload information may be hard to come by while doing the initial design of the system. Consequently, tuning a database after it has been designed and deployed is important—we must refine the initial design in the light of actual usage patterns to obtain the best possible performance.

The distinction between database design and database tuning is somewhat arbitrary. We could consider the design process to be over once an initial conceptual schema is designed and a set of indexing and clustering decisions is made. Any subsequent changes to the conceptual schema or the indexes, say, would then be regarded as tuning. Alternatively, we could consider some refinement of the conceptual schema (and physical design decisions affected by this refinement) to be part of the physical design process.

Where we draw the line between design and tuning is not very important, and we simply discuss the issues of index selection and database tuning without regard to when the tuning is carried out.

## **20.2 GUIDELINES FOR INDEX SELECTION**

In considering which indexes to create, we begin with the list of queries (including queries that appear as part of update operations). Obviously, only relations accessed by some query need to be considered as candidates for indexing, and the choice of attributes to index is guided by the conditions that appear in the WHERE clauses of the queries in the workload. The presence of suitable indexes can significantly improve the evaluation plan for a query, as we saw in Chapters 8 and 12.

One approach to index selection is to consider the most important queries in turn, and, for each, determine which plan the optimizer would choose given the indexes currently on our list of (to be created) indexes. Then we consider whether we can arrive at a substantially better plan by adding more indexes; if so, these additional indexes are candidates for inclusion in our list of indexes. In general, range retrievals benefit from a B+ tree index, and exact-match retrievals benefit from a hash index. Clustering benefits range queries, and it benefits exact-match queries if several data entries contain the same key value.

Before adding an index to the list, however, we must consider the impact of having this index on the updates in our workload. As we noted earlier, although an index can speed up the query component of an update, all indexes on an updated attribute—on *any* attribute, in the case of inserts and deletes—must be updated whenever the value of the attribute is changed. Therefore, we must sometimes consider the trade-off of slowing some update operations in the workload in order to speed up some queries.

Clearly, choosing a good set of indexes for a given workload requires an understanding of the available indexing techniques, and of the workings of the query optimizer. The following guidelines for index selection summarize our discussion:

**Whether to Index (Guideline 1):** The obvious points are often the most important. Do not build an index unless some query—including the query components of updates—benefits from it. Whenever possible, choose indexes that speed up more than one query.

**Choice of Search Key (Guideline 2):** Attributes mentioned in a WHERE clause are candidates for indexing.

- An exact-match selection condition suggests that we consider an index on the selected attributes, ideally, a hash index.
- A range selection condition suggests that we consider a B+ tree (or ISAM) index on the selected attributes. A B+ tree index is usually preferable to an ISAM index. An ISAM index may be worth considering if the relation is infrequently updated, but we assume that a B+ tree index is always chosen over an ISAM index, for simplicity.

**Multi-Attribute Search Keys (Guideline 3):** Indexes with multiple-attribute search keys should be considered in the following two situations:

- A WHERE clause includes conditions on more than one attribute of a relation.

- They enable index-only evaluation strategies (i.e., accessing the relation can be avoided) for important queries. (This situation could lead to attributes being in the search key even if they do not appear in WHERE clauses.)

When creating indexes on search keys with multiple attributes, if range queries are expected, be careful to order the attributes in the search key to match the queries.

**Whether to Cluster (Guideline 4):** At most one index on a given relation can be clustered, and clustering affects performance greatly; so the choice of clustered index is important.

- As a rule of thumb, range queries are likely to benefit the most from clustering. If several range queries are posed on a relation, involving different sets of attributes, consider the selectivity of the queries and their relative frequency in the workload when deciding which index should be clustered.
- If an index enables an index-only evaluation strategy for the query it is intended to speed up, the index need not be clustered. (Clustering matters only when the index is used to retrieve tuples from the underlying relation.)

**Hash versus Tree Index (Guideline 5):** A B+ tree index is usually preferable because it supports range queries as well as equality queries. A hash index is better in the following situations:

- The index is intended to support index nested loops join; the indexed relation is the inner relation, and the search key includes the join columns. In this case, the slight improvement of a hash index over a B+ tree for equality selections is magnified, because an equality selection is generated for each tuple in the outer relation.
- There is a very important equality query, and no range queries, involving the search key attributes.

**Balancing the Cost of Index Maintenance (Guideline 6):** After drawing up a 'wishlist' of indexes to create, consider the impact of each index on the updates in the workload.

- If maintaining an index slows down frequent update operations, consider dropping the index.
- Keep in mind, however, that adding an index may well speed up a given update operation. For example, an index on employee IDs could speed up the operation of increasing the salary of a given employee (specified by ID).



## 20.3 BASIC EXAMPLES OF INDEX SELECTION

The following examples illustrate how to choose indexes during database design, continuing the discussion from Chapter 8, where we focused on index selection for single-table queries. The schemas used in the examples are not described in detail; in general, they contain the attributes named in the queries. Additional information is presented when necessary.

Let us begin with a simple query:

```
SELECT E.ename, D.dname
FROM   Employees E, Departments D
WHERE  D.dname='Toy' AND E.dno=D.dno
```

The relations mentioned in the query are *Employees* and *Departments*, and both conditions in the *WHERE* clause involve equalities. Our guidelines suggest that we should build hash indexes on the attributes involved. It seems clear that we should build a hash index on the *dname* attribute of *Departments*. But consider the equality *E.dno=D.dno*. Should we build an index (hash, of course) on the *dno* attribute of *Departments* or *Employees* (or both)? Intuitively, we want to retrieve *Departments* tuples using the index on *dname* because few tuples are likely to satisfy the equality selection *D.dname='Toy'*.<sup>1</sup> For each qualifying *Departments* tuple, we then find matching *Employees* tuples by using an index on the *dno* attribute of *Employees*. So, we should build an index on the *dno* field of *Employees*. (Note that nothing is gained by building an additional index on the *dno* field of *Departments* because *Departments* tuples are retrieved using the *dname* index.)

Our choice of indexes was guided by the query evaluation plan we wanted to utilize. This consideration of a potential evaluation plan is common while making physical design decisions. Understanding query optimization is very useful for physical design. We show the desired plan for this query in Figure 20.1.

As a variant of this query, suppose that the *WHERE* clause is modified to be *WHERE D.dname='Toy' AND E.dno=D.dno AND E.age=25*. Let us consider alternative evaluation plans. One good plan is to retrieve *Departments* tuples that satisfy the selection on *dname* and retrieve matching *Employees* tuples by using an index on the *dno* field; the selection on *age* is then applied on-the-fly. However, unlike the previous variant of this query, we do not really need to have an index on the *dno* field of *Employees* if we have an index on *age*. In this

---

<sup>1</sup>This is only a heuristic. If *dname* is not the key, and we have no statistics to verify this claim, it is possible that several tuples satisfy this condition.



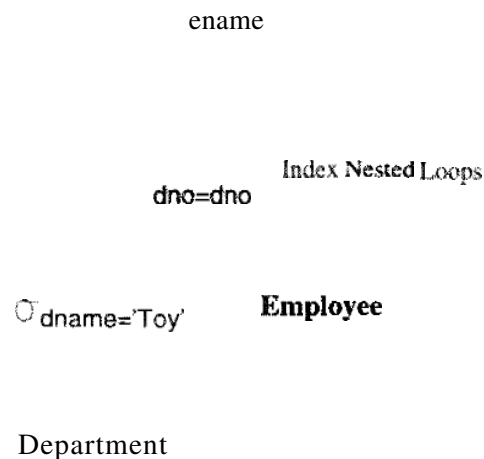


Figure 20.1 A Desirable Query Evaluation Plan

case we can retrieve Departments tuples that satisfy the selection on *dname* (by using the index on *dname*, as before), retrieve Employees tuples that satisfy the selection on *age* by using the index on *age*, and join these sets of tuples. Since the sets of tuples we join are small, they fit in memory and the join method is unimportant. This plan is likely to be somewhat poorer than using an index on *dno*, but it is a reasonable alternative. Therefore, if we have an index on *age* already (prompted by some other query in the workload), this variant of the sample query does not justify creating an index on the *dno* field of Employees.

Our next query involves a range selection:

```

SELECT E.ename, I.dname
FROM   Employees E, Departments D
WHERE  E.sal BETWEEN 10000 AND 20000
       AND E.hobby='Starups' AND E.dno=D.dno
  
```

This query illustrates the use of the BETWEEN operator for expressing range selections. It is equivalent to the condition:

$$10000 \leq E.sal \text{ AND } E.sal \leq 20000$$

The use of BETWEEN to express range conditions is recommended; it makes it easier for both the user and the optimizer to recognize both parts of the range selection.

Returning to the example query, both (nonjoin) selections are on the Employees relation. Therefore, it is clear that a plan in which Employees is the outer relation and Departments is the inner relation is the best, as in the previous query, and we should build a hash index on the *dno* attribute of Departments. But which index should we build on Employees? A B+ tree index on the *sal* attribute would help with the range selection, especially if it is clustered. A

hash index on the *hobby* attribute would help with the equality selection. If one of these indexes is available, we could retrieve Employees tuples using this index, retrieve matching Departments tuples using the index on *dno*, and apply all remaining selections and projections on-the-fly. If both indexes are available, the optimizer would choose the more selective index for the given query; that is, it would consider which selection (the range condition on *salary* or the equality on *hobby*) has fewer qualifying tuples. In general, which index is more selective depends on the data. If there are very few people with salaries in the given range and many people collect stamps, the B-tree index is best. Otherwise, the hash index on *hobby* is best.

If the query constants are known (as in our example), the selectivities can be estimated if statistics on the data are available. Otherwise, as a rule of thumb, an equality selection is likely to be more selective, and a reasonable decision would be to create a hash index on *hobby*. Sometimes, the query constants are not known—we might obtain a query by expanding a query on a view at runtime, or we might have a query in Dynamic SQL, which allows constants to be specified as *wild-card variables* (e.g., %X) and instantiated at runtime (see Sections 6.1.3 and 6.2). In this case, if the query is very important, we might choose to create a B+ tree index on *sal* and a hash index on *hobby* and leave the choice to be made by the optimizer at runtime.

## 20.4 CLUSTERING AND INDEXING

Clustered indexes can be especially important while accessing the inner relation in an index nested loops join. To understand the relationship between clustered indexes and joins, let us revisit our first example:

```
SELECT E.ename, D.dname
FROM   Employees E, Departments D
WHERE  D.dname='Toy' AND E.dno=D.dno
```

We concluded that a good evaluation plan is to use an index on *dname* to retrieve Departments tuples satisfying the condition on *dname* and to find matching Employees tuples using an index on *dno*. Should these indexes be clustered? Given our assumption that the number of tuples satisfying 1). *dname*='Toy' is likely to be small, we should build an unclustered index on *dname*. On the other hand, Employees is the inner relation in an index nested loops join and *dno* is not a candidate key. This situation is a strong argument that the index on the *dno* field of Employees should be clustered. In fact, because the join consists of repeatedly posing equality selections on the *dno* field of the inner relation, this type of query is a stronger justification for making the index on *dno* clustered than a simple selection query such as the previous selection on

*hobby*. (Of course, factors such as selectivities and frequency of queries have to be taken into account as well.)

The following example, very similar to the previous one, illustrates how clustered indexes can be used for sort-merge joins:

```
SELECT E.ename,D.dname
FROM   Employees E, Departments D
WHERE  E.hobby='Stamps' AND E.dno=D.dno
```

This query differs from the previous query in that the condition *E.hobby*='Stamps' replaces *D.dname*='Toy'. Based on the assumption that there are few employees in the Toy department, we chose indexes that would facilitate an indexed nested loops join with Departments as the outer relation. Now, let us suppose that many employees collect stamps. In this case, a block nested loops or sort-merge join might be more efficient. A sort-merge join can take advantage of a clustered B+ tree index on the *dno* attribute in Departments to retrieve tuples and thereby avoid sorting Departments. Note that an unclustered index is not useful---since all tuples are retrieved, performing one I/O per tuple is likely to be prohibitively expensive. If there is no index on the *dno* field of Employees, we could retrieve Employees tuples (possibly using an index on *hobby*, especially if the index is clustered), apply the selection *E.hobby*='Stamps' on-the-fly, and sort the qualifying tuples on *dno*.

As our discussion has indicated, when we retrieve tuples using an index, the impact of clustering depends on the number of retrieved tuples, that is, the number of tuples that satisfy the selection conditions that match the index. An unclustered index is just as good as a clustered index for a selection that retrieves a single tuple (e.g., an equality selection on a candidate key). As the number of retrieved tuples increases, the unclustered index quickly becomes more expensive than even a sequential scan of the entire relation. Although the sequential scan retrieves all tuples, each page is retrieved exactly once, whereas a page may be retrieved as often as the number of tuples it contains if an unclustered index is used. If blocked I/O is performed (as is common), the relative advantage of sequential scan versus an unclustered index increases further. (Blocked I/O also speeds up access using a clustered index, of course.)

We illustrate the relationship between the number of retrieved tuples, viewed as a percentage of the total number of tuples in the relation, and the cost of various access methods in Figure 20.2. We assume that the query is a selection on a single relation, for simplicity. (Note that this figure reflects the cost of writing out the result; otherwise, the line for sequential scan would be flat.)

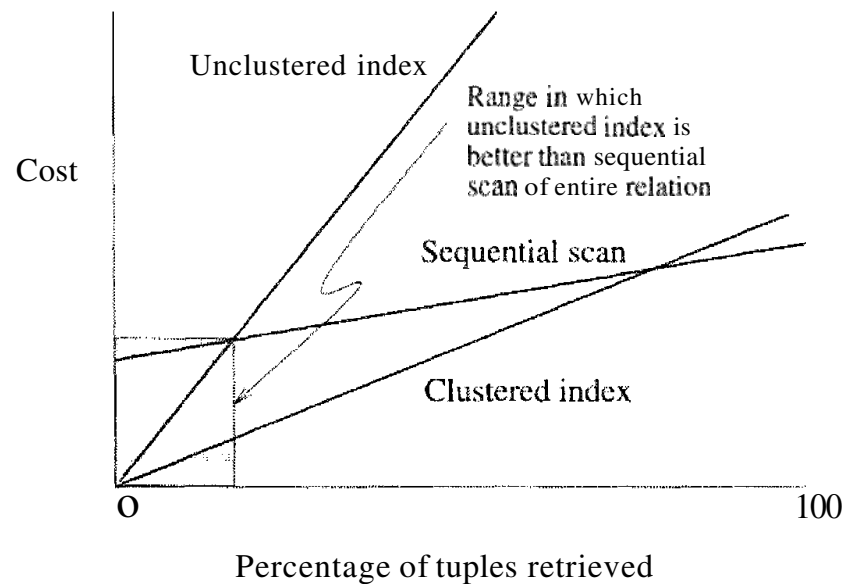


Figure 20.2 The Impact of Clustering

### 20.4.1 Co-clustering Two Relations

In our description of a typical database system architecture in Chapter 9, we explained how a relation is stored as a file of records. Although a file usually contains only the records of *some* one relation, *some* systems allow records from *more* than one relation to be stored in a single file. *Some* database user can request that the records from *two* relations be interleaved physically in this *file*. This data layout is *sometimes* referred to as co-clustering the two relations. We now discuss when co-clustering can be beneficial.

As an example, consider two relations with the following schemas:

```
Parts(pid: integer, name: string, cost: integer, supplierid: integer)
Assembly(partid: integer, componentid: integer, quantity: integer)
```

In this schema the *componentid* field of Assembly is intended to be the *pid* of *some* part that is used as a component in assembling the part with *pid* equal to *partid*. Therefore, the Assembly table represents a 1:N relationship between parts and their subparts; a part can have *many* subparts, but each part is the subpart of at most one part. In the Parts table, *pid* is the key. For composite parts (those assembled from other parts, as indicated by the contents of Assembly), the *cost* field is taken to be the cost of assembling the part from its subparts.

Suppose that a frequent query is to find the (immediate) subparts of all parts supplied by a given supplier:

```
SELECT P.pid, A.componentid
FROM   Parts P, Assembly A
```

```
WHERE P.pid = A.partid AND P.supplierid = 'Acme'
```

A good evaluation plan is to apply the selection condition on Parts and then retrieve matching Assembly tuples through an index on the *partid* field. Ideally, the index on *partid* should be clustered. This plan is reasonably good. However, if such selections are common and we want to optimize them further, we can *co-cluster* the two tables. In this approach, we store records of the two tables together, with each Parts record *P* followed by all the Assembly records *A* such that *P.pid = A.partid*. This approach improves on storing the two relations separately and having a clustered index on *partid* because it does not need an index lookup to find the Assembly records that match a given Parts record. Thus, for each selection query, we save a few (typically two or three) index page I/Os.

If we are interested in finding the immediate subparts of *all* parts (i.e., the preceding query with no selection on *supplierid*), creating a clustered index on *partid* and doing an index nested loops join with Assembly as the inner relation offers good performance. An even better strategy is to create a clustered index on the *partid* field of Assembly and the *pid* field of Parts, then do a sort-merge join, using the indexes to retrieve tuples in sorted order. This strategy is comparable to doing the join using a co-clustered organization, which involves just one scan of the set of tuples (of Parts and Assembly, which are stored together in interleaved fashion).

The real benefit of co-clustering is illustrated by the following query:

```
SELECT P.pid, A.componentid
FROM   Parts P, Assembly A
WHERE  P.pid = A.partid AND P.cost=10
```

Suppose that many parts have *cost* = 10. This query essentially amounts to a collection of queries in which we are given a Parts record and want to find matching Assembly records. If we have an index on the *cost* field of Parts, we can retrieve qualifying Parts tuples. For each such tuple, we have to use the index on Assembly to locate records with the given *pid*. The index access for Assembly is avoided if we have a co-clustered organization. (Of course, we still require an index on the *cost* attribute of Parts tuples.)

Such an optimization is especially important if we want to traverse several levels of the part-subpart hierarchy. For example, a common query is to find the total cost of a part, which requires us to repeatedly carry out joins of Parts and Assembly. Incidentally, if we do not know the number of levels in the hierarchy in advance, the number of joins varies and the query cannot be expressed in SQL. The query can be answered by embedding an SQL statement

for the join inside an iterative host language program. How to express the query is orthogonal to our main point here, which is that co-clustering is especially beneficial when the join in question is carried out very frequently (either because it arises repeatedly in an important query such as finding total cost, or because the join query itself is asked frequently).

To summarize co-clustering:

- It can speed up joins, in particular key-foreign key joins corresponding to 1:N relationships.
- A sequential scan of either relation becomes slower. (In our example, since several Assembly tuples are stored in between consecutive Parts tuples, a scan of all Parts tuples becomes slower than if Parts tuples were stored separately. Similarly, a sequential scan of all Assembly tuples is also slower.)
- All inserts, deletes, and updates that alter record lengths become slower, thanks to the overheads involved in maintaining the clustering. (We do not discuss the implementation issues involved in co-clustering.)

## 20.5 INDEXES THAT ENABLE INDEX-ONLY PLANS

This section considers a number of queries for which we can find efficient plans that avoid retrieving tuples from one of the referenced relations; instead, these plans scan an associated index (which is likely to be much smaller). An index that is used (only) for index-only scans does *not* have to be clustered because tuples from the indexed relation are not retrieved.

This query retrieves the managers of departments with at least one employee:

```
SELECT D.mgr
FROM   Departments D, Employees E
WHERE  D.dno=E.dno
```

Observe that no attributes of Employees are retained. If we have an index on the *dno* field of Employees, the optimization of doing an index nested loops join using an index-only search for the inner relation is applicable. Given this variant of the query, the correct decision is to build an unclustered index on the *dno* field of Employees, rather than a clustered index.

The next query takes this idea a step further:

```
SELECT D.mgr, E.ename
FROM   Departments D, Employees E
WHERE  D.dno=E.dno
```

If we have an index on the *dno* field of *Employees*, we can use it to retrieve *Employees* tuples during the join (with *Departments* as the outer relation), but unless the index is clustered, this approach is not be efficient. On the other hand, suppose that we have a B+- tree index on  $(dno, e'id)$ . Now all the information we need about an *Employees* tuple is contained in the data entry for this tuple in the index. We can use the index to find the first data entry with a given *dno*; all data entries with the same *dno* are stored together in the index. (Note that a hash index on the composite key  $(dno, eid)$  cannot be used to locate an entry with just a given *dno*!) We can therefore evaluate this query using an index nested loops join with *Departments* as the outer relation and an index-only scan of the inner relation.

## 20.6 TOOLS TO ASSIST IN INDEX SELECTION

The number of possible indexes to consider building is potentially very large: For each relation, we can potentially consider all possible subsets of attributes as an index key; we have to decide on the ordering of the attributes in the index; and we also have to decide which indexes should be clustered and which unclustered. Many large applications—for example enterprise resource planning systems—create tens of thousands of different relations, and manual tuning of such a large schema is a daunting endeavor.

The difficulty and importance of the index selection task motivated the development of tools that help database administrators select appropriate indexes for a given workload. The first generation of such index tuning wizards, or index advisors, were separate tools outside the database engine; they suggested indexes to build, given a workload of SQL queries. The main drawback of these systems was that they had to replicate the database query optimizer's cost model in the tuning tool to make sure that the optimizer would choose the same query evaluation plans as the design tool. Since query optimizers change from release to release of a commercial database system, considerable effort was needed to keep the tuning tool and the database optimizer synchronized. The most recent generation of tuning tools are integrated with the database engine and use the database query optimizer to estimate the cost of a workload given a set of indexes, avoiding duplication of the query optimizer's cost model into an external tool.

### 20.6.1 Automatic Index Selection

We call a set of indexes for a given database schema an index configuration. We assume that a query workload is a set of queries over a database schema where each query has a frequency of occurrence assigned to it. Given a database schema and a workload, the cost of an index configuration is the expected



cost of running the queries in the workload given the index configuration taking the different frequencies of queries in the workload into account. Given a database schema and a query workload, we can now define the problem of automatic index selection as finding an index configuration with minimal cost. As in query optimization, in practice our goal is to find a *good* index configuration rather than the true optimal configuration.

Why is automatic index selection a hard problem? Let us calculate the number of different indexes with  $c$  attributes, assuming that the table has  $n$  attributes. For the first attribute in the index, there are  $n$  choices, for the second attribute  $n-1$ , and thus for a  $c$  attribute index, there are overall  $n \cdot (n-1) \dots (n-c+1) = \frac{n!}{(n-c)!}$  different indexes possible. The total number of different indexes with up to  $c$  attributes is

$$\sum_{i=1}^c \frac{n!}{(n-i)!}$$

For a table with 10 attributes, there are 10 different one-attribute indexes, 90 different two-attribute indexes, and 30240 different five-attribute indexes. For a complex workload involving hundreds of tables, the number of possible index configurations is clearly very large.

The efficiency of automatic index selection tools can be separated into two components: (1) the number of candidate index configurations considered, and (2) the number of optimizer calls necessary to evaluate the cost for a configuration. Note that reducing the search space of candidate indexes is analogous to restricting the search space of the query optimizer to left-deep plans. In many cases, the optimal plan is not left-deep, but among all left-deep plans there is usually a plan whose cost is close to the optimal plan.

We can easily reduce the time taken for automatic index selection by reducing the number of candidate index configurations, but the smaller the space of index configurations considered, the farther away the final index configuration is from the optimal index configuration. Therefore, different index tuning wizards prune the search space differently, for example, by considering only one- or two-attribute indexes.

## 20.6.2 How Do Index Tuning Wizards Work?

All index tuning wizards search a set of candidate indexes for an index configuration with lowest cost. Tools differ in the space of candidate index configurations they consider and how they search this space. We describe one representative algorithm; existing tools implement variants of this algorithm, but their implementations have the same basic structure.



**The DB2 Index Advisor.** The DB2 Index Advisor is a tool for automatic index recommendation given a workload. The workload is stored in the database system in a table called `ADVISE_WORKLOAD`. It is populated either (1) by SQL statements from the DB2 dynamic SQL statement cache, a cache for recently executed SQL statements, (2) with SQL statements from packages—groups of statically compiled SQL statements, or (3) with SQL statements from an online monitor called the Query Patroller. The DB2 Advisor allows the user to specify the maximum amount of disk space for new indexes and a maximum time for the computation of the recommended index configuration.

The DB2 Index Advisor consists of a program that intelligently searches a subset of index configurations. Given a candidate configuration, it calls the query optimizer for each query in the `ADVISE_WORKLOAD` table first in the `RECOMMEND_INDEXES` mode, where the optimizer recommends a set of indexes and stores them in the `ADVISE_INDEXES` table. In the `EVALUATE_INDEXES` mode, the optimizer evaluates the benefit of the index configuration for each query in the `ADVISE_WORKLOAD` table. The output of the index tuning step is a set of SQL DDL statements whose execution creates the recommended indexes.

**The Microsoft SQL Server 2000 Index Tuning Wizard.** Microsoft pioneered the implementation of a tuning wizard integrated with the database query optimizer. The Microsoft Index Tuning Wizard has three tuning modes that permit the user to trade off running time of the analysis and number of candidate index configurations examined: *fast*, *medium*, and *thorough*, with *fast* having the lowest running time and *thorough* examining the largest number of configurations. To further reduce the running time, the tool has a sampling mode in which the tuning wizard randomly samples queries from the input workload to speed up analysis. Other parameters include the maximum space allowed for the recommended indexes, the maximum number of attributes per index considered, and the tables on which indexes can be generated. The Microsoft Index Tuning Wizard also permits *table scaling*, where the user can specify an anticipated number of records for the tables involved in the workload. This allows users to plan for future growth of the tables.

Before we describe the index tuning algorithm, let us consider the problem of estimating the cost of a configuration. Note that it is not feasible to actually create the set of indexes in a candidate configuration and then optimize the query workload given the physical index configuration. Creation of even a single candidate configuration with several indexes might take hours for large databases and put considerable load on the database system itself. Since we want to examine a large number of possible candidate configurations, this approach is not feasible.

Therefore index tuning algorithms usually *simulate* the effect of indexes in a candidate configuration (unless such indexes already exist). Such what-if indexes look to the query optimizer like any other index and are taken into account when calculating the cost of the workload for a given configuration, but the creation of what-if indexes does not incur the overhead of actual index creation. Commercial database systems that support index tuning wizards using the database query optimizer have been extended with a module that permits the creation and deletion of what-if indexes with the necessary statistics about the indexes (that are used when estimating the cost of a query plan).

We now describe a representative index tuning algorithm. The algorithm proceeds in two steps, *candidate index selection* and *configuration enumeration*. In the first step, we select a set of candidate indexes to consider during the second step as building blocks for index configurations. Let us discuss these two steps in more detail.

## Candidate Index Selection

We saw in the previous section that it is impossible to consider every possible index, due to the huge number of candidate indexes available for larger database schemas. One heuristic to prune the large space of possible indexes is to first tune each query in the workload independently and then select the union of the indexes selected in this first step as input to the second step.

For a query, let us introduce the notion of an indexable attribute, which is an attribute whose appearance in an index could change the cost of the query. An indexable attribute is an attribute on which the WHERE-part of the query has a condition (e.g., an equality predicate) or the attribute appears in a GROUP BY or ORDER BY clause of the SQL query. An admissible index for a query is an index that contains only indexable attributes in the query.

How do we select candidate indexes for an individual query? One approach is a basic enumeration of all indexes with up to  $k$  attributes. We start with all indexable attributes as single attribute candidate indexes, then add all com-

binations of two indexable attributes as candidate indexes, and repeat this procedure until a user-defined size threshold  $k$ . This procedure is obviously very expensive as we add overall  $n + n \cdot (n - 1) + \dots + n \cdot (n - 1) \dots (n - k + 1)$  candidate indexes, but it guarantees that the best index with up to  $k$  attributes is among the candidate indexes. The references at the end of this chapter contain pointers to faster (but less exhaustive) heuristic search algorithms.

## Enumerating Index Configurations

In the second phase, we use the candidate indexes to enumerate index configurations. As in the first phase, we can exhaustively enumerate all index configurations up to size  $k$ , this time combining candidate indexes. As in the previous phase, more sophisticated search strategies are possible that cut down the number of configurations considered while still generating a final configuration of high quality (i.e., low execution cost for the final workload).

## 20.7 OVERVIEW OF DATABASE TUNING

After the initial phase of database design, actual use of the database provides a valuable source of detailed information that can be used to refine the initial design. Many of the original assumptions about the expected workload can be replaced by observed usage patterns; in general, some of the initial workload specification is validated, and some of it turns out to be wrong. Initial guesses about the size of data can be replaced with actual statistics from the system catalogs (although this information keeps changing as the system evolves). Careful monitoring of queries can reveal unexpected problems; for example, the optimizer may not be using some indexes as intended to produce good plans.

Continued database tuning is important to get the best possible performance. In this section, we introduce three kinds of tuning: *tuning indexes*, *tuning the conceptual schema*, and *tuning queries*. Our discussion of index selection also applies to index tuning decisions. Conceptual schema and query tuning are discussed further in Sections 20.8 and 20.9.

### 20.7.1 Tuning Indexes

The initial choice of indexes may be refined for one of several reasons. The simplest reason is that the observed workload reveals that some queries and updates considered important in the initial workload specification are not very frequent. The observed workload may also identify some new queries and updates that *are* important. The initial choice of indexes has to be reviewed in light of this new information. Some of the original indexes may be dropped and

new ones added. The reasoning involved is similar to that used in the initial design.

It may also be discovered that the optimizer in a given system is not finding some of the plans that it was expected to. For example, consider the following query, which we discussed earlier:

```
SELECT D.dname
FROM   Employees E, Departments D
WHERE  D.dname='Toy' AND E.dno=D.dno
```

A good plan here would be to use an index on *dname* to retrieve Departments tuples with *dname* = 'Toy' and to use an index on the *dno* field of Employees as the inner relation, using an index-only scan. Anticipating that the optimizer would find such a plan, we might have created an unclustered index on the *dno* field of Employees.

Now suppose queries of this form take an unexpectedly long time to execute. We can ask to see the plan produced by the optimizer. (Most commercial systems provide a simple command to do this.) If the plan indicates that an index-only scan is not being used, but that Employees tuples are being retrieved, we have to rethink our initial choice of index, given this revelation about our system's (unfortunate) limitations. An alternative to consider here would be to drop the unclustered index on the *dno* field of Employees and replace it with a clustered index.

Some other common limitations of optimizers are that they do not handle selections involving string expressions, arithmetic, or *null* values effectively. We discuss these points further when we consider query tuning in Section 20.9.

In addition to re-examining our choice of indexes, it pays to periodically reorganize static indexes. For example, a static index, such as an ISAM index, may have developed long overflow chains. Dropping the index and rebuilding it—if feasible, given the interrupted access to the indexed relation—can substantially improve access times through this index. Even for a dynamic structure such as a B+ tree, if the implementation does not merge pages on deletes, space occupancy can decrease considerably in some situations. This in turn makes the size of the index (in pages) larger than necessary, and could increase the height and therefore the access time. Rebuilding the index should be considered. Extensive updates to a clustered index might also lead to overflow pages being allocated, thereby decreasing the degree of clustering. Again, rebuilding the index may be worthwhile.

Finally, note that the query optimizer relies on statistics maintained in the SystCIII catalogs. These statistics are updated only when a special utility program is run; be sure to run the utility frequently enough to keep the statistics reasonably current.

## 20.7.2 Tuning the Conceptual Schema

In the course of database design, we may realize that our current choice of relation SchemaS does not enable us meet our performance objectives for the given workload with any (feasible) set of physical design choices. If so, we may have to redesign our conceptual schema (and re-examine physical design decisions affected by the changes we make).

We may realize that a redesign is necessary during the initial design process or later, after the system has been in use for a while. Once a database has been designed and populated with tuples, changing the conceptual schema requires a significant effort in terms of mapping the contents of the relations affected. Nonetheless, it may be necessary to revise the conceptual schema in light of experience with the system. (Such changes to the schema of an operational system are sometimes referred to as schema evolution.) We now consider the issues involved in conceptual schema (re)design from the point of view of performance.

The main point to understand is that *our choice of conceptual schema should be guided by a consideration of the queries and 'updates' in our 'workload'* in addition to the issues of redundancy that motivate nonnormalization (which we discussed in Chapter 19). Several options must be considered while tuning the conceptual schema:

- We may decide to settle for a 3NF design instead of a BCNF design.
- If there are two ways to decompose a given schema into 3NF or BCNF, our choice should be guided by the workload.
- Sometimes we might decide to further decompose a relation that is *already* in BCNF.
- In other situations, we might *denormalize*. That is, we might choose to replace a collection of relations obtained by a decomposition from a larger relation with the original (larger) relation, even though it suffers from some redundancy problems. Alternatively, we might choose to add some fields to certain relations to speed up some important queries, even if this leads to a redundant storage of some information (and, consequently, a schema that is in neither 3NF nor BCNF).

- This discussion of nonnalization has concentrated on the technique of *decomposition*, which amounts to vertical partitioning of a relation. Another technique to consider is *horizontal partitioning* of a relation, which would lead to having two relations with identical schemas. Note that we are not talking about physically partitioning the tuples of a single relation; rather, we want to create two distinct relations (possibly with different constraints and indexes on each).

Incidentally, when we redesign the conceptual schema, especially if we are tuning an existing database schema, it is worth considering whether we should create views to mask these changes from users for whom the original schema is more natural. We discuss the choices involved in tuning the conceptual schema in Section 20.8.

### 20.7.3 Tuning Queries and Views

If we notice that a query is running much slower than we expected, we have to examine the query carefully to find the problem. Some rewriting of the query, perhaps in conjunction with some index tuning, can often fix the problem. Similar tuning may be called for if queries on some view run slower than expected. We do not discuss view tuning separately; just think of queries on views as queries in their own right (after all, queries on views are expanded to account for the view definition before being optimized) and consider how to tune them.

When tuning a query, the first thing to verify is that the system uses the plan you expect it to use. Perhaps the system is not finding the best plan for a variety of reasons. Some common situations not handled efficiently by many optimizers follow:

- A selection condition involving *null* values.
- Selection conditions involving arithmetic or string expressions or conditions using the OR connective. For example, if we have a condition  $E.age = 2 * J.age$  in the WHERE clause, the optimizer may correctly utilize an available index on  $E.age$  but fail to utilize an available index on  $J.age$ . Replacing the condition by  $E.age / 2 = J.age$  would reverse the situation.
- Inability to recognize a sophisticated plan such as an index-only scan for an aggregation query involving a GROUP BY clause. Of course, virtually no optimizer looks for plans outside the plan space described in Chapters 12 and 15, such as nonleft-deep join trees. So a good understanding of what an optimizer typically does is important. In addition, the more aware you are of a given system's strengths and limitations, the better off you are.

If the optimizer is not smart enough to find the best plan (using access methods and evaluation strategies supported by the DBMS), some systems allow users to guide the choice of a plan by providing hints to the optimizer; for example, users might be able to force the use of a particular index or choose the join order and join method. A user who wishes to guide optimization in this manner should have a thorough understanding of both optimization and the capabilities of the given DBMS. We discuss query tuning further in Section 20.9.

## 20.8 CHOICES IN TUNING THE CONCEPTUAL SCHEMA

We now illustrate the choices involved in tuning the conceptual schema through several examples using the following schemas:

```
Contracts(cid: integer, supplierid: integer, projectid: integer,
          deptid: integer, partid: integer, qty: integer, value: real)
Departments(did: integer, budget: real, annualreport: varchar)
Parts(pid: integer, cost: integer)
Projects(jid: integer, mng: char(20))
Suppliers(sid: integer, address: char(50))
```

For brevity, we often use the common convention of denoting attributes by a single character and denoting relation schemas by a sequence of characters. Consider the schema for the relation *Contracts*, which we denote as CSJDPQV, with each letter denoting an attribute. The meaning of a tuple in this relation is that the contract with *cid* C is an agreement that supplier S (with *sid* equal to *supplierid*) will supply Q items of part P (with *pid* equal to *partid*) to project J (with *jid* equal to *projectid*) associated with department D (with *deptid* equal to *did*), and that the value V of this contract is equal to *value*.<sup>2</sup>

There are two known integrity constraints with respect to *Contracts*. A project purchases a given part using a single contract; thus, there cannot be two distinct contracts in which the same project buys the same part. This constraint is represented using the FD  $JP \rightarrow C$ . Also, a department purchases at most one part from any given supplier. This constraint is represented using the FD  $SD \rightarrow P$ . In addition, of course, the contract ID C is a key. The meaning of the other relations should be obvious, and we do not describe them further because we focus on the *Contracts* relation.

---

<sup>2</sup>If this schema seems complicated, note that real-life situations often call for considerably more complex schemas!



### 20.8.1 Settling for a Weaker Normal Form

Consider the Contracts relation. Should we decompose it into smaller relations? Let us see what normal form it is in. The candidate keys for this relation are C and JP. (C is given to be a key, and JP functionally determines C.) The only nonkey dependency is  $SD \rightarrow P$ , and P is a *prime* attribute because it is part of candidate key JP. Thus, the relation is not in BCNF—because there is a nonkey dependency—but it is in 3NF.

By using the dependency  $SD \rightarrow P$  to guide the decomposition, we get the two schemas SDP and CSJDQV. This decomposition is lossless, but it is not dependency-preserving. However, by adding the relation scheme CJP, we obtain a lossless-join, dependency-preserving decomposition into BCNF. Using the guideline that such a decomposition into BCNF is good, we might decide to replace Contracts by three relations with schemas CJP, SDP, and CSJDQV.

However, suppose that the following query is very frequently asked: Find the number of copies Q of part P ordered in contract C. This query requires a join of the decomposed relations CJP and CSJDQV (or SDP and CSJDQV), whereas it can be answered directly using the relation Contracts. The added cost for this query could persuade us to settle for a 3NF design and not decompose Contracts further.

### 20.8.2 Denormalization

The reasons motivating us to settle for a weaker normal form may lead us to take an even more extreme step: deliberately introduce some redundancy. As an example, consider the Contracts relation, which is in 3NF. Now, suppose that a frequent query is to check that the value of a contract is less than the budget of the contracting department. We might decide to add a budget field B to Contracts. Since *did* is a key for Departments, we now have the dependency  $D \rightarrow B$  in Contracts, which means Contracts is not in 3NF any more. Nonetheless, we might choose to stay with this design if the motivating query is sufficiently important. Such a decision is clearly subjective and comes at the cost of significant redundancy.

### 20.8.3 Choice of Decomposition

Consider the Contracts relation again. Several choices are possible for dealing with the redundancy in this relation:

- We can leave Contracts as it is and accept the redundancy associated with its being in 3NF rather than BCNF.



- We might decide that we want to avoid the anomalies resulting from this redundancy by decomposing Contracts into BCNF using one of the following methods:
  - We have a lossless-join decomposition into PartInfo with attributes SDP and ContractInfo with attributes CSJDQV. As noted previously, this decomposition is not dependency-preserving, and to make it so would require us to add a third relation CJP, whose sole purpose is to allow us to check the dependency  $JP \rightarrow C$ .
  - We could choose to replace Contracts by just PartInfo and ContractInfo even though this decomposition is not dependency-preserving.

Replacing Contracts by just PartInfo and ContractInfo does not prevent us from enforcing the constraint  $JP \rightarrow C$ ; it only makes this more expensive. We could create an assertion in SQL-92 to check this constraint:

```
CREATE ASSERTION checkDep
CHECK      ( NOT EXISTS
            (SELECT *
              FROM      PartInfo PI, ContractInfo CI
              WHERE      PI.supplierid=CI.supplierid
                        AND PI.deptid=CI.deptid
              GROUP BY  CI.projectid, PI.partid
              HAVING     COUNT (cid) > 1 ) )
```

This assertion is expensive to evaluate because it involves a join followed by a sort (to do the grouping). In comparison, the system can check that  $JP$  is a primary key for table CJP by maintaining an index on  $JP$ . This difference in integrity-checking cost is the motivation for dependency-preservation. On the other hand, if updates are infrequent, this increased cost may be acceptable; therefore, we might choose not to maintain the table CJP (and quite likely, an index on it).

As another example illustrating decomposition choices, consider the Contracts relation again, and suppose that we also have the integrity constraint that a department uses a given supplier for at most one of its projects:  $SPQ \rightarrow V$ . Proceeding as before, we have a lossless-join decomposition of Contracts into SDP and CSJDQV. Alternatively, we could begin by using the dependency  $SPQ \rightarrow V$  to guide our decomposition, and replace Contracts with SPQV and CSJDPQ. We can then decompose CSJDPQ, guided by  $SD \rightarrow P$ , to obtain SDP and CSJDQ.

We now have two alternative lossless-join decompositions of Contracts into BCNF, neither of which is dependency-preserving. The first alternative is to

replace **Contracts** with the relations **SDP** and **CSJDQV**. The second alternative is to replace it with **SPQV**, **SDP**, and **CSJDQ**. The addition of **CJP** makes the second decomposition (but not the first) dependency-preserving. Again, the cost of maintaining the three relations **CJP**, **SPQV**, and **CSJDQ** (versus just **CSJDQV**) may lead us to choose the first alternative. In this case, enforcing the given FDs becomes more expensive. We might consider not enforcing them, but we then risk a violation of the integrity of our data.

### 20.8.4 Vertical Partitioning of BCNF Relations

Suppose that we have decided to decompose **Contracts** into **SDP** and **CSJDQV**. These schemas are in BCNF, and there is no reason to decompose them further from a normalization standpoint. However, suppose that the following queries are very frequent:

- Find the contracts held by supplier **S**.
- Find the contracts placed by department **D**.

These queries might lead us to decompose **CSJDQV** into **CS**, **CD**, and **CJQV**. The decomposition is lossless, of course, and the two important queries can be answered by examining much smaller relations. Another reason to consider such a decomposition is concurrency control *hot spots*. If these queries are common, and the most common updates involve changing the quantity of products (and the value) involved in contracts, the decomposition improves performance by reducing lock contention. Exclusive locks are now set mostly on the **CJQV** table, and reads on **CS** and **CD** do not conflict with these locks.

Whenever we decompose a relation, we have to consider which queries the decomposition might adversely affect, especially if the only motivation for the decomposition is improved performance. For example, if another important query is to find the total value of contracts held by a supplier, it would involve a join of the decomposed relations **CS** and **CJQV**. In this situation, we might decide against the decomposition.

### 20.8.5 Horizontal Decomposition

Thus far, we have essentially considered how to replace a relation with a collection of vertical decompositions. Sometimes, it is worth considering whether to replace a relation with two relations that have the same attributes as the original relation, each containing a subset of the tuples in the original. Intuitively, this technique is useful when different subsets of tuples are queried in very distinct ways.

For example, different rules may govern large contracts, which are defined as contracts with values greater than 10,000. (Perhaps, such contracts have to be awarded through a bidding process.) This constraint could lead to a number of queries in which Contracts tuples are selected using a condition of the form  $value > 10,000$ . One way to approach this situation is to build a clustered B+ tree index on the *value* field of Contracts. Alternatively, we could replace Contracts with two relations called LargeContracts and SmallContracts, with the obvious meaning. If this query is the only motivation for the index, horizontal decomposition offers all the benefits of the index without the overhead of index maintenance. This alternative is especially attractive if other important queries on Contracts also require clustered indexes (on fields other than *value*).

If we replace Contracts by two relations LargeContracts and SmallContracts, we could mask this change by defining a view called Contracts:

```
CREATE VIEW Contracts(cid, supplierid, projectid, deptid, partid, qty, value)
AS ((SELECT *
    FROM LargeContracts)
    UNION
    (SELECT *
    FROM SmallContracts))
```

However, any query that deals solely with LargeContracts should be expressed directly on LargeContracts and not on the view. Expressing the query on the view Contracts with the selection condition  $value > 10,000$  is equivalent to expressing the query on LargeContracts but less efficient. This point is quite general: Although we can mask changes to the conceptual schema by adding view definitions, users concerned about performance have to be aware of the change.

As another example, if Contracts had an additional field *year* and queries typically dealt with the contracts in some one year, we might choose to partition Contracts by year. Of course, queries that involved contracts from more than one year might require us to pose queries against each of the decomposed relations.

## 20.9 CHOICES IN TUNING QUERIES AND VIEWS

The first step in tuning a query is to understand the plan used by the DBMS to evaluate the query. Systems usually provide some facility for identifying the plan used to evaluate a query. Once we understand the plan selected by the system, we can consider how to improve performance. We can consider a different choice of indexes or perhaps co-clustering two relations for join queries,

guided by our understanding of the old plan and a better plan that we want the DBVIS to use. The details are similar to the initial design process.

One point worth making is that before creating new indexes we should consider whether rewriting the query achieves acceptable results with existing indexes. For example, consider the following query with an OR connective:

```
SELECT E.dno
FROM   Employees E
WHERE  E.hobby='Sports' OR E.age==10
```

If we have indexes on both *hobby* and *age*, we can use these indexes to retrieve the necessary tuples, but an optimizer might fail to recognize this opportunity. The optimizer might view the conditions in the WHERE clause as a whole as not matching either index, do a sequential scan of Employees, and apply the selections on-the-fly. Suppose we rewrite the query as the union of two queries, one with the clause *WHERE E.hobby = 'Sports'* and the other with the clause *WHERE E.age==10*. Now each query is answered efficiently with the aid of the indexes on *hobby* and *age*.

We should also consider rewriting the query to avoid some expensive operations. For example, including DISTINCT in the SELECT clause leads to duplicate elimination, which can be costly. Thus, we should omit DISTINCT whenever possible. For example, for a query on a single relation, we can omit DISTINCT whenever either of the following conditions holds:

- We do not care about the presence of duplicates.
- The attributes mentioned in the SELECT clause include a candidate key for the relation.

Sometimes a query with GROUP BY and HAVING can be replaced by a query without these clauses, thereby eliminating a sort operation. For example, consider:

```
SELECT  MIN (E.age)
FROM    Employees E
GROUP BY E.dno
HAVING  E.dno=102
```

This query is equivalent to

```
SELECT  MIN (E.age)
FROM    Employees E
WHERE   E.dno=102
```

Complex queries are often written in steps, using a temporary relation. We can usually rewrite such queries without the temporary relation to make them run faster. Consider the following query for computing the average salary of departments managed by Robinson:

```
SELECT  *
INTO    Ternp
FROM    EmployeesE, Departments D
WHERE   E.dno=D.dno AND D.mgrname='Robinson'
```

```
SELECT  T.dno, AVG (T.sal)
FROM    Ternp T
GROUP BY T.dno
```

This query can be rewritten as

```
SELECT  E.dno, AVG (E.sal)
FROM    Employees E, Departments D
WHERE   E.dno=D.dno AND D.mgrname='Robinson'
GROUP BY E.dno
```

The rewritten query does not materialize the intermediate relation, Ternp and is therefore likely to be faster. In fact, the optimizer may even find a very efficient index-only plan that never retrieves Employees tuples if there is a composite B+ tree index on (dno, sal). This example illustrates a general observation: *By rewriting queries to avoid unnecessary temporaries, we not only avoid creating the temporary relations, we also open up more optimization possibilities for the optimizer to explore.*

In some situations, however, if the optimizer is unable to find a good plan for a complex query (typically a nested query with correlation), it may be worthwhile to rewrite the query using temporary relations to guide the optimizer toward a good plan.

In fact, nested queries are a common source of inefficiency because many optimizers deal poorly with them, as discussed in Section 15.5. Whenever possible, it is better to rewrite a nested query without nesting and a correlated query without correlation. As already noted, a good reformulation of the query may require us to introduce new, temporary relations, and techniques to do so systematically (ideally, to be done by the optimizer) have been widely studied. (Often though, it is possible to rewrite nested queries without nesting or the use of temporary relations, as illustrated in Section 15.5.

## 20.10 IMPACT OF CONCURRENCY

In a system with many concurrent users, several additional points must be considered. Transactions obtain *locks* on the pages they access, and other transactions may be blocked waiting for locks on objects they wish to access.

We observed in Section 16.5 that blocking delays must be minimized for good performance and identified two specific ways to reduce blocking:

- Reducing the time that transactions hold locks.
- Reducing hot spots.

We now discuss techniques for achieving these goals.

### 20.10.1 Reducing Lock Durations

**Delay Lock Requests:** Tune transactions by writing to local program variables and deferring changes to the database until the end of the transaction. This delays the acquisition of the corresponding locks and reduces the time the locks are held.

**Make Transactions Faster:** The sooner a transaction completes, the sooner its locks are released. We have already discussed several ways to speed up queries and updates (e.g., using indexes, rewriting queries). In addition, a careful partitioning of the tuples in a relation and its associated indexes across a collection of disks can significantly improve concurrent access. For example, if we have the relation on one disk and an index on another, accesses to the index can proceed without interfering with accesses to the relation, at least at the level of disk reads.

**Replace Long Transactions by Short Ones:** Sometimes, just too much work is done within a transaction, and it takes a long time and holds locks a long time. Consider rewriting the transaction as two or more smaller transactions; holdable cursors (see Section 6.1.2) can be helpful in doing this. The advantage is that each new transaction completes quicker and releases locks sooner. The disadvantage is that the original list of operations is no longer executed atomically, and the application code must deal with situations in which one or more of the new transactions fail.

**Build a Warehouse:** Complex queries can hold shared locks for a long time. Often, however, these queries involve statistical analysis of business trends and it is acceptable to run them on a copy of the data that is a little out of date. This led to the popularity of *data warehouses*, which are databases that complement

the operational database by maintaining a copy of data used in complex queries (Chapter 25). Running these queries against the warehouse relieves the burden of long-running queries from the operational database.

**Consider a Lower Isolation Level:** In many situations, such as queries generating aggregate information or statistical summaries, we can use a lower SQL isolation level such as REPEATABLE READ or READ COMMITTED (Section 16.6). Lower isolation levels incur lower locking overheads, and the application programmer must make good design trade-offs.

## 20.10.2 Reducing Hot Spots

**Delay Operations on Hot Spots:** We already discussed the value of delaying lock requests. Obviously, this is especially important for requests involving frequently used objects.

**Optimize Access Patterns:** The pattern of updates to a relation can also be significant. For example, if tuples are inserted into the Employees relation in *eid* order and we have a B+ tree index on *eid*, each insert goes to the last leaf page of the B+ tree. This leads to hot spots along the path from the root to the rightmost leaf page. Such considerations may lead us to choose a hash index over a B+ tree index or to index on a different field. Note that this pattern of access leads to poor performance for ISAM indexes as well, since the last leaf page becomes a hot spot. This is not a problem for hash indexes because the hashing process randomizes the bucket into which a record is inserted.

**Partition Operations on Hot Spots:** Consider a data entry transaction that appends new records to a file (e.g., inserts into a table stored as a heap file). Instead of appending records one-per-transaction and obtaining a lock on the last page for each record, we can replace the transaction by several other transactions, each of which writes records to a local file and periodically appends a batch of records to the main file. While we do more work overall, this reduces the lock contention on the last page of the original file.

As a further illustration of partitioning, suppose we track the number of records inserted in a counter. Instead of updating this counter once per record, the preceding approach results in updating several counters and periodically updating the main counter. This idea can be adapted to many uses of counters, with varying degrees of effort. For example, consider a counter that tracks the number of reservations, with the rule that a new reservation is allowed only if the counter is below a maximum value. We can replace this by three counters, each with one-third the original maximum threshold, and three transactions that use these counters rather than the original. We obtain greater concurrency, but



have to deal with the case where one of the counters is at the maximum value but some other counter can still be incremented. Thus, the price of greater concurrency is increased complexity in the logic of the application code.

**Choice of Index:** If a relation is updated frequently, B+ tree indexes can become a concurrency control bottleneck, because all accesses through the index must go through the root. Thus, the root and index pages just below it can become hot spots. If the DBMS uses specialized locking protocols for tree indexes, and in particular, sets fine-granularity locks, this problem is greatly alleviated. Many current systems use such techniques.

Nonetheless, this consideration may lead us to choose an ISAM index in some situations. Because the index levels of an ISAM index are static, we need not obtain locks on these pages; only the leaf pages need to be locked. An ISAM index may be preferable to a B+ tree index, for example, if frequent updates occur but we expect the relative distribution of records and the number (and size) of records with a given range of search key values to stay approximately the same. In this case the ISAM index offers a lower locking overhead (and reduced contention for locks), and the distribution of records is such that few overflow pages are created.

Hashed indexes do not create such a concurrency bottleneck, unless the data distribution is very skewed and many data items are concentrated in a few buckets. In this case, the directory entries for these buckets can become a hot spot.

## 20.11 CASE STUDY: THE INTERNET SHOP

Revisiting our running case study, DBDudes considers the expected workload for the B&N Bookstore. The owner of the bookstore expects most of his customers to search for books by ISBN number before placing an order. Placing an order involves inserting one record into the Orders table and inserting one or more records into the Orderlists relation. If a sufficient number of books is available, a shipment is prepared and a value for the *ship\_date* in the Orderlists relation is set. In addition, the available quantities of books in stock changes all the time, since orders are placed that decrease the quantity available and new books arrive from suppliers and increase the quantity available.

The DBDudes team begins by considering searches for books by ISBN. Since *isbn* is a key, an equality query on *isbn* returns at most one record. Therefore, to speed up queries from customers who look for books with a given ISBN, DBDudes decides to build an unclustered hash index on *isbn*.



Next, it considers updates to book quantities. To update the *qty\_in\_stock* value for a book, we must first search for the book by ISBN; the index on *isbn* speeds this up. Since the *qty\_in\_stock* value for a book is updated quite frequently, DBDudes also considers partitioning the Books relation vertically into the following two relations:

```
BooksQty(isbn, qty)
BookRest(isbn, title, author, price, year_published)
```

Unfortunately, this vertical partitioning slows down another very popular query: Equality search on ISBN to retrieve all information about a book now requires a join between BooksQty and BookRest. So DBDudes decides not to vertically partition Books.

DBDudes thinks it is likely that customers will also want to search for books by title and by author, and decides to add unclustered hash indexes on *title* and *author*—these indexes are inexpensive to maintain because the set of books is rarely changed even though the quantity in stock for a book changes often.

Next, DBDudes considers the Customers relation. A customer is first identified by the unique customer identification number. So the most common queries on Customers are equality queries involving the customer identification number, and DBDudes decides to build a clustered hash index on *cid* to achieve maximum speed for this query.

Moving on to the Orders relation, DBDudes sees that it is involved in two queries: insertion of new orders and retrieval of existing orders. Both queries involve the *ordernum* attribute as search key and so DBDudes decides to build an index on it. What type of index should this be—a B+ tree or a hash index? Since order numbers are assigned sequentially and correspond to the order date, sorting by *ordernum* effectively sorts by order date as well. So DBDudes decides to build a clustered B+ tree index on *ordernum*. Although the operational requirements mentioned until now favor neither a B+ tree nor a hash index, B&N will probably want to monitor daily activities and the clustered B+ tree is a better choice for such range queries. Of course, this means that retrieving all orders for a given customer could be expensive for customers with many orders, since clustering by *ordernum* precludes clustering by other attributes, such as *cid*.

The Orderlists relation involves mostly insertions, with an occasional update of a shipment date or a query to list all components of a given order. If Orderlists is kept sorted on *ordernum*, all insertions are appends at the end of the relation and thus very efficient. A clustered B+ tree index on *ordernum* maintains this sort order and also speeds up retrieval of all items for a given order. To update

a shipment date, we need to search for a tuple by *ordernum* and *isbn*. The index on *ordernum* helps here as well. Although an index on  $\langle \textit{ordernum}, \textit{isbn} \rangle$  would be better for this purpose, insertions would not be as efficient as with an index on just *ordernum*; DBDudes therefore decides to index *Orderlists* on just *ordernum*.

### 20.11.1 Tuning the Database

Several months after the launch of the B&N site, DBDudes is called in and told that customer enquiries about pending orders are being processed very slowly. B&N has become very successful, and the *Orders* and *Orderlists* tables have grown huge.

Thinking further about the design, DBDudes realizes that there are two types of orders: *completed orders*, for which all books have already shipped, and *partially completed orders*, for which some books are yet to be shipped. Most customer requests to look up an order involve partially completed orders, which are a small fraction of all orders. DBDudes therefore decides to horizontally partition both the *Orders* table and the *Orderlists* table by *ordernum*. This results in four new relations: *NewOrders*, *OldOrders*, *NewOrderlists*, and *OldOrderlists*.

An order and its components are always in exactly one pair of relations—and we can determine which pair, old or new, by a simple check on *ordernum*—and queries involving that order can always be evaluated using only the relevant relations. Some queries are now slower, such as those asking for all of a customer's orders, since they require us to search two sets of relations. However, these queries are infrequent and their performance is acceptable.

## 20.12 DBMS BENCHMARKING

Thus far, we considered how to improve the design of a database to obtain better performance. As the database grows, however, the underlying DBMS may no longer be able to provide adequate performance, even with the best possible design, and we have to consider upgrading our system, typically by buying faster hardware and additional memory. We may also consider migrating our database to a new DBMS.

When evaluating DBMS products, performance is an important consideration. ADBIVIS is a complex piece of software, and different vendors may target their systems toward different market segments by putting more effort into optimizing certain parts of the system or choosing different system designs. For example, some systems are designed to run complex queries efficiently, while others are designed to run many simple transactions per second. Within

each category of systems, there are many competing products. To assist users in choosing a DBMS that is well suited to their needs, several performance benchmarks have been developed. These include benchmarks for measuring the performance of a certain class of applications (e.g., the TPC benchmarks) and benchmarks for measuring how well a DBMS performs various operations (e.g., the Wisconsin benchmark).

Benchmarks should be portable, easy to understand, and scale naturally to larger problem instances. They should measure *peak performance* (e.g., *transactions per second*, or *tps*) as well as *price/performance ratios* (e.g., *\$/tps*) for typical workloads in a given application domain. The Transaction Processing Council (TPC) was created to define benchmarks for transaction processing and database systems. Other well-known benchmarks have been proposed by academic researchers and industry organizations. Benchmarks that are proprietary to a given vendor are not very useful for comparing different systems (although they may be useful in determining how well a given system would handle a particular workload).

## 20.12.1 Well-Known DBMS Benchmarks

**Online Transaction Processing Benchmarks:** The TPC-A and TPC-B benchmarks constitute the standard definitions of the *tps* and *\$/tps* measures. TPC-A measures the performance and price of a computer network in addition to the DBMS, whereas the TPC-B benchmark considers the DBMS by itself. These benchmarks involve a simple transaction that updates three data records, from three different tables, and appends a record to a fourth table. A number of details (e.g., transaction arrival distribution, interconnect method, system properties) are rigorously specified, ensuring that results for different systems can be meaningfully compared. The TPC-C benchmark is a more complex suite of transactional tasks than TPC-A and TPC-B. It models a warehouse that tracks items supplied to customers and involves five types of transactions. Each TPC-C transaction is much more expensive than a TPC-A or TPC-B transaction, and TPC-C exercises a much wider range of system capabilities, such as use of secondary indexes and transaction aborts. It has more or less completely replaced TPC-A and TPC-B as the standard transaction processing benchmark.

**Query Benchmarks:** The Wisconsin benchmark is widely used for measuring the performance of simple relational queries. The Set Query benchmark measures the performance of a suite of more complex queries, and the *AS<sup>3</sup>AP* benchmark measures the performance of a mixed workload of transactions, relational queries, and utility functions. The TPC-D benchmark is a suite of complex SQL queries intended to be representative of the (decision-support ap-

plication domain. The OLAP Council also developed a benchmark for complex decision-support queries, including some queries that cannot be expressed easily in SQL; this is intended to measure systems for *online analytic processing (OLAP)*, which we discuss in Chapter 25, rather than traditional SQL systems. The Sequoia 2000 benchmark is designed to compare DBMS support for geographic information systems.

**Object-Database Benchmarks:** The OO1 and OO7 benchmarks measure the performance of object-oriented database systems. The Bucky benchmark measures the performance of object-relational database systems. (We discuss object-database systems in Chapter 23.)

## 20.12.2 Using a **Benchmark**

Benchmarks should be used with a good understanding of what they are designed to measure and the application environment in which a DBMS is to be used. When you use benchmarks to guide your choice of a DBMS, keep the following guidelines in mind:

- **How Meaningful is a Given Benchmark?** Benchmarks that try to distill performance into a single number can be overly simplistic. A DBMS is a complex piece of software used in a variety of applications. A good benchmark should have a suite of tasks that are carefully chosen to cover a particular application domain and test DBMS features important for that domain.
- **How Well Does a Benchmark Reflect Your Workload?** Consider your expected workload and compare it with the benchmark. Give little weight to the performance of those benchmark tasks (i.e., queries and updates) that are similar to important tasks in your workload. Also consider how benchmark numbers are measured. For example, elapsed time for individual queries might be misleading if considered in a multiuser setting: A system may have higher elapsed times because of slower I/O. On a multiuser workload, given sufficient disks for parallel I/O, such a system might outperform a system with a lower elapsed time.
- **Create Your Own Benchmark:** Vendors often tweak their systems in ad hoc ways to obtain good numbers on important benchmarks. To counter this, create your own benchmark by modifying standard benchmarks slightly or by replacing the tasks in a standard benchmark with similar tasks from your workload.

## 20.13 REVIEW QUESTIONS

Answers to the review questions can be found in the listed sections.

- || What are the components of a workload description? (Section 20.1.1)
- What decisions need to be made during physical design? (Section 20.1.2)
- || Describe six high-level guidelines for index selection. (Section 20.2)
- When should we create clustered indexes? (Section 20.4)
- What is co-clustering, and when should we use it? (Section 20.4.1)
- || What is an index-only plan, and how do we create indexes for index-only plans? (Section 20.5)
- Why is automatic index tuning a hard problem? Give an example. (Section 20.6.1)
- Give an example of one algorithm for automatic index tuning. (Section 20.6.2)
- || Why is database tuning important? (Section 20.7)
- || How do we tune indexes, the conceptual schema, and queries and views? (Sections 20.7.1 to 20.7.3)
- What are our choices in tuning the conceptual schema? What are the following techniques and when should we apply them: settling for a weaker normal form, denormalization, and horizontal and vertical decompositions. (Section 20.8)
- What choices do we have in tuning queries and views? (Section 20.9)
- What is the impact of locking on database performance? How can we reduce lock contention and hot spots? (Section 20.10)
- Why do we have standardized database benchmarks, and what common metrics are used to evaluate database systems? Can you describe a few popular database benchmarks? (Section 20.12)

## EXERCISES

Exercise 20.1 Consider the following BCNF schema for a portion of a simple corporate database (type information is not relevant to this question and is omitted):

Emp (eid, *ename*, *addr*, *sal*, *age*, *yrs*, *deptid*)  
 Dept (did, *dname*, *flood*, *budget*)

Suppose you know that the following queries are the six most common queries in the workload for this corporation and that all six are roughly equivalent in frequency and importance:

- List the id, name, and address of employees in a user-specified age range.
- List the id, name, and address of employees who work in the department with a user-specified department name.
- List the id and address of employees with a user-specified employee name.
- List the overall average salary for employees.
- List the average salary for employees of each age; that is, for each age in the data, list the age and the corresponding average salary.
- List all the department information, ordered by department floor numbers.

1. Given this information, and assuming that these queries are more important than any updates, design a physical schema for the corporate database that will give good performance for the expected workload. In particular, decide which attributes will be indexed and whether each index will be a clustered index or an unclustered index. Assume that B+ tree indexes are the only index type supported by the DBMS and that both single- and multiple-attribute keys are permitted. Specify your physical design by identifying the attributes you recommend indexing on via clustered or unclustered B+ trees.
2. Redesign the physical schema assuming that the set of important queries is changed to be the following:
  - List the id and address of employees with a user-specified employee name.
  - List the overall maximum salary for employees.
  - List the average salary for employees by department; that is, for each *deptid* value, list the *deptid* value and the average salary of employees in that department.
  - List the sum of the budgets of all departments by floor; that is, for each floor, list the floor and the sum.
  - Assume that this workload is to be tuned with an automatic index tuning wizard. Outline the main steps in the execution of the index tuning algorithm and the set of candidate configurations that would be considered.

**Exercise 20.2** Consider the following BCNF<sup>1</sup> relational schema for a portion of a university database (type information is not relevant to this question and is omitted):

*Prof*(*ssno*, *pname*, *office*, *age*, *sex*, *specialty*, *dept\_did*)  
*Dept*(*did*, *dname*, *budget*, *num\_majors*, *chair\_ssno*)

Suppose you know that the following queries are the five most common queries in the workload for this university and that all five are roughly equivalent in frequency and importance:

- List the names, ages, and offices of professors of a user-specified sex (male or female) who have a user-specified research specialty (e.g., *recursive query processing*). Assume that the university has a diverse set of faculty members, making it very uncommon for more than a few professors to have the same research specialty.
- List all the department information for departments with professors in a user-specified age range.
- List the department id, department name, and chairperson name for departments with a user-specified number of majors.

- List the lowest budget for a department in the university.
- List all the information about professors who are department chairpersons.

These queries occur much more frequently than updates, so you should build whatever indexes you need to speed up these queries. However, you should not build any unnecessary indexes, as updates will occur (and would be slowed down by unnecessary indexes). Given this information, design a physical schema for the university database that will give good performance for the expected workload. In particular, decide which attributes should be indexed and whether each index should be a clustered index or an unclustered index. Assume that both B+ trees and hashed indexes are supported by the DBMS and that both single- and multiple-attribute index search keys are permitted.

1. Specify your physical design by identifying the attributes you recommend indexing on, indicating whether each index should be clustered or unclustered and whether it should be a B+ tree or a hashed index.
2. Assume that this workload is to be tuned with an automatic index tuning wizard. Outline the main steps in the algorithm and the set of candidate configurations considered.
3. Redesign the physical schema, assuming that the set of important queries is changed to be the following:
  - List the number of different specialties covered by professors in each department, by department.
  - Find the department with the fewest majors.
  - Find the youngest professor who is a department chairperson.

**Exercise 20.3** Consider the following BCNF relational schema for a portion of a company database (type information is not relevant to this question and is omitted):

*Project*(*pno*, *proj\_name*, *proj\_base\_dept*, *proj\_mgr*, *topic*, *budget*)  
*Manager*(*mid*, *mgr\_name*, *mgr\_dept*, *salary*, *age*, *sex*)

Note that each project is based in some department, each manager is employed in some department, and the manager of a project need not be employed in the same department (in which the project is based). Suppose you know that the following queries are the five most common queries in the workload for this university and all five are roughly equivalent in frequency and importance:

- List the names, ages, and salaries of managers of a user-specified sex (male or female) working in a given department. You can assume that, while there are many departments, each department contains very few project managers.
- List the names of all projects with managers whose ages are in a user-specified range (e.g., younger than 30).
- List the names of all departments such that a manager in this department manages a project based in this department.
- List the name of the project with the lowest budget.
- List the names of all managers in the same department as a given project.

These queries occur much more frequently than updates, so you should build whatever indexes you need to speed up these queries. However, you should not build any unnecessary indexes, as updates will occur (and would be slowed down by unnecessary indexes). Given



this information, design a physical schema for the company database that will give good performance for the expected workload. In particular, decide which attributes should be indexed and whether each index should be a clustered index or an unclustered index. Assume that both B+ trees and hashed indexes are supported by the DBMS, and that both single- and multiple-attribute index keys are permitted.

1. Specify your physical design by identifying the attributes you recommend indexing on, indicating whether each index should be clustered or unclustered and whether it should be a B+ tree or a hashed index.
2. Assume that this workload is to be tuned with an automatic index tuning wizard. Outline the main steps in the algorithm and the set of candidate configurations considered.
3. Redesign the physical schema assuming the set of important queries is changed to be the following:
  - Find the total of the budgets for projects managed by each manager; that is, list *proj\_mgr* and the total of the budgets of projects managed by that manager, for all values of *proj\_mgr*.
  - Find the total of the budgets for projects managed by each manager but only for managers who are in a user-specified age range.
  - Find the number of male managers.
  - Find the average age of managers.

**Exercise 20.4** The Globetrotters Club is organized into chapters. The president of a chapter can never serve as the president of any other chapter, and each chapter gives its president some salary. Chapters keep moving to new locations, and a new president is elected when (and only when) a chapter moves. This data is stored in a relation  $G(C, S, L, P)$ , where the attributes are chapters ( $C$ ), salaries ( $S$ ), locations ( $L$ ), and presidents ( $P$ ). Queries of the following form are frequently asked, and you must be able to answer them without computing a join: “Who was the president of chapter  $X$  when it was in location  $Y$ ?”

1. List the FDs that are given to hold over  $G$ .
2. What are the candidate keys for relation  $G$ ?
3. What normal form is the schema  $G$  in?
4. Design a good database schema for the club. (Remember that your design *must* satisfy the stated query requirement!)
5. What normal form is your good schema in? Give an example of a query that is likely to run slower on this schema than on the relation  $G$ .
6. Is there a lossless-join, dependency-preserving decomposition of  $G$  into BCNF?
7. Is there ever a good reason to accept something less than 3NF when designing a schema for a relational database? Use this example, if necessary adding further constraints, to illustrate your answer.

**Exercise 20.5** Consider the following BCNF relation, which lists the ids, types (e.g., nuts or bolts), and costs of various parts, along with the number available or in stock:

Parts (*pid*, *pname*, *cost*, *num\_avail*)

You are told that the following two queries are extremely important:



## Physical Database Design and Tuning

- Find the total number available by part type, for all types. (That is, the sum of the *num\_avail* value of all nuts, the sum of the *num\_avail* value of all bolts, and so forth)
  - List the *pids* of parts with the highest cost.
1. Describe the physical design that you would choose for this relation. That is, what kind of a file structure would you choose for the set of Parts records, and what indexes would you create?
  2. Suppose your customers subsequently complain that performance is still not satisfactory (given the indexes and file organization you chose for the Parts relation in response to the previous question). Since you cannot afford to buy new hardware or software, you have to consider a schema redesign. Explain how you would try to obtain better performance by describing the schema for the relation(s) that you would use and your choice of file organizations and indexes on these relations.
  3. How would your answers to the two questions change, if at all, if your system did not support indexes with multiple-attribute search keys?

Exercise 20.6 Consider the following BCNF relations, which describe employees and the departments they work in:

Emp (eid, sal, did)  
Dept (d'id, location, budget)

You are told that the following queries are extremely important:

- Find the location where a user-specified employee works.
  - Check whether the budget of a department is greater than the salary of each employee in that department.
1. Describe the physical design you would choose for this relation. That is, what kind of a file structure would you choose for these relations, and what indexes would you create?
  2. Suppose that your customers subsequently complain that performance is still not satisfactory (given the indexes and file organization that you chose for the relations in response to the previous question). Since you cannot afford to buy new hardware or software, you have to consider a schema redesign. Explain how you would try to obtain better performance by describing the schema for the relation(s) that you would use and your choice of file organizations and indexes on these relations.
  3. Suppose that your database system has very inefficient implementations of index structures. What kind of a design would you try in this case?

Exercise 20.7 Consider the following BCNF relations, which describe departments in a company and employees:

Dept(did, dname, location, managerid)  
Emp(eid, sal)

You are told that the following queries are extremely important:

- List the names and ids of managers for each department in a user-specified location, in alphabetical order by department name.
- Find the average salary of employees who manage departments in a user-specified location. You can assume that no one manages more than one department.

1. Describe the file structures and indexes that you would choose.
2. You subsequently realize that updates to these relations are frequent. Because indexes incur a high overhead, can you think of a way to improve performance on these queries without using indexes?

**Exercise 20.8** For each of the following queries, identify one possible reason why an optimizer might not find a good plan. Rewrite the query so that a good plan is likely to be found. Any available indexes or known constraints are listed before each query; assume that the relation schemas are consistent with the attributes referred to in the query.

1. An index is available on the *age* attribute:

```
SELECT E.dno
FROM   Employee E
WHERE  E.age=20 OR E.age=10
```

2. A B+ tree index is available on the *age* attribute:

```
SELECT E.dno
FROM   Employee E
WHERE  E.age<20 AND E.age>10
```

3. An index is available on the *age* attribute:

```
SELECT E.eIno
FROM   Employee E
WHERE  2*E.age<20
```

4. No index is available:

```
SELECT DISTINCT *
FROM   Employee E
```

5. No index is available:

```
SELECT  AVG (B.sal)
FROM    Employee E
GROUP BY E.dno
HAVING  E.dno=22
```

6. The *sid* in Reserves is a foreign key that refers to Sailors:

```
SELECT  S.sid
FROM    Sailors S, Reserves H
WHERE   S.sid=R.sid
```

**Exercise 20.9** Consider two ways to compute the names of employees who earn more than \$100,000 and whose age is equal to their manager's age. First, a nested query:

```
SELECT  E1.ename
FROM    Emp E1
WHERE   E1.sal > 100 AND E1.age = ( SELECT E2.age
                                   FROM   Emp E2, Dept D2
                                   WHERE  E1.dname = D2.dname
                                   AND    D2.mgr = E2.ename )
```

Second, a query that uses a view definition:

```

SELECT  E1.ename
FROM    Emp E1, MgrAge A
WHERE   E1.dname = A.dname AND E1.sal > 100 AND E1.age = A.age

```

```

CREATE VIEW MgrAge (dname, age)
AS SELECT D.dname, E.age
   FROM   Emp E, Dept D
   WHERE  D.mgr = E.ename

```

1. Describe a situation in which the first query is likely to outperform the second query.
2. Describe a situation in which the second query is likely to outperform the first query.
3. Can you construct an equivalent query that is likely to beat both these queries when every employee who earns more than \$100,000 is either 35 or 40 years old? Explain briefly.

## BIBLIOGRAPHIC NOTES

[658] is an early discussion of physical database design. [659] discusses the performance implications of normalization and observes that denormalization may improve performance for certain queries. The ideas underlying a physical design tool from IBM are described in [272]. The Microsoft AutoAdmin tool that performs automatic index selection according to a query workload is described in several papers [163, 164]. The DB2 Advisor is described in [750]. Other approaches to physical database design are described in [146, 639]. [679] considers *transaction tuning*, which we discussed only briefly. The issue is how an application should be structured into a collection of transactions to maximize performance.

The following books on database design cover physical design issues in detail; they are recommended for further reading. [274] is largely independent of specific products, although many examples are based on DB2 and Teradata systems. [779] deals primarily with DB2. Shasha and Bonnet give an in-depth, readable introduction to database tuning [104].

[334] contains several papers on benchmarking database systems and has accompanying software. It includes articles on the *AS<sup>3</sup>AP*, Set Query, TPC-A, TPC-B, Wisconsin, and OO1 benchmarks written by the original developers. The Bucky benchmark is described in [132], the OO1 benchmark is described in [131], and the TPC-D benchmark is described in [739]. The Sequoia 2000 benchmark is described in [720].